

Phase 6: Performance Evaluation and Analysis (Week 6):

1. Document your approach:

Documented Approach (Methodology):

Applying the apps chosen in Week 3 an Ubuntu hosting 24.04 stable machine served to a speed test. The goal was to look at how the operating system acted in different load settings. To be positive, it lined up with proper server admin steps, testing was done by way of SSH.

Four cases were used for testing each application:

- . System idle with no extra workload during baseline testing
- . Applications actively producing load are tested.
- . Analysing difficulties to identify power shortage
- . Applying changes and fresh improvement checking

Using common Linux tools, system performance metrics were collected:

- . For both RAM and CPU, use peak and htop.
- . Free h to use memory
- . disc I/O via df – h or iostat.
- . To assess network performance, use iperf3 and ping.
- . Output of application commands for measuring response time.

2. Create a performance data table with structured measurements for all applications and metrics?

Because they define basic OS features, several OS system-wide variables have been chosen.

. Use of CPUs

The success of planning app processes and their final job load is tracked by tracking CPU use.

. Consumption of Memory

Memory use was studied to find out buffer usage, storage capacity, and possible disc pressure.

Disc I/O Efficiency In order learn about file system output as well data time, the disc writes and reads action has been studied.

. Network Efficiency

The value of exchanges between a computer and server was measured by analysing network capacity and delays.

. The system's latency

The delays in performing duties with increased load were used for measuring the speed with which the system was.

. **Service Response Times**

Response times were used to measure how quickly services reacted to requests during load conditions.

3.Create performance visualisations including charts and graphs?

A) Baseline Performance Testing

The baseline test was carried out with minimal workload conditions to set normal system behaviour standards. In this test phase, the system had low CPU and memory utilization, minimal disk I/O, as well as consistent network latency.

b) Application Load Testing

Application load testing entailed the simulation of the workloads related to services that are CPU intensive, memory intensive, disk intensive, and network based. At the increased load, the simulation was done based on high CPU utilization, memory use, disk use, and network use. This case tested the priority scheduling of processes by the operating system.

c) Performance Analysis & Bottleneck Identification

Analysing the gathered data helped in pointing out the existence of various performance limitations for which the

The increased use of the CPU resulted in increased system latency.

The caching efficiency was affected by memory pressure.

This makes file access slower since there would be more disk-seeking operations.

Network-intensive workloads saw latency go up and throughput go down

These bottlenecks revealed just how critical resource allocation and management are within an operating system.

d) Optimization Testing

Two optimization techniques were applied and compared:

Service Hardening and Resource Limitation

Background services in Windows, if unnecessary, had been switched off.

Enhanced Security and Scheduling Efficiency The configurations for security-related areas like firewalls and access control have also been optimized to remove unnecessary processing overheads. Measurements made after the optimization testing has been completed showed improvements in system responsiveness.

4.Capture testing evidence

CPU & Memory Testing:

<Enter> to resume, filters: none											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
52	root	20	0	0	0	0	I	0.3	0.0	0:04.21	kworker/2:1-e
938	root	20	0	936784	46524	27908	S	0.3	1.2	0:02.14	fail2ban-ser
1	root	20	0	22020	12896	9312	S	0.0	0.3	0:14.36	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.08	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workque
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_g
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slj
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-net
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.03	kworker/0:0H
11	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/u6:0
12	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mn
13	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_kth
14	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_rud
15	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_tra
16	root	20	0	0	0	0	S	0.0	0.0	0:00.55	ksoftirqd/0
17	root	20	0	0	0	0	I	0.0	0.0	0:00.78	rcu_preempt
18	root	rt	0	0	0	0	S	0.0	0.0	0:00.05	migration/0
19	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/0
20	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
21	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
22	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/1
23	root	rt	0	0	0	0	S	0.0	0.0	0:01.14	migration/1
24	root	20	0	0	0	0	S	0.0	0.0	0:00.16	ksoftirqd/1
25	root	20	0	0	0	0	I	0.0	0.0	0:00.46	kworker/1:0-events
27	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/2
28	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/2
29	root	rt	0	0	0	0	S	0.0	0.0	0:01.18	migration/2
30	root	20	0	0	0	0	S	0.0	0.0	0:01.42	ksoftirqd/2
31	root	20	0	0	0	0	I	0.0	0.0	0:00.88	kworker/2:0-cgroup_destroy
32	root	0	-20	0	0	0	I	0.0	0.0	0:00.16	kworker/2:0H-kblockd
33	root	20	0	0	0	0	I	0.0	0.0	0:00.16	kworker/u7:0-flush-252:0
36	root	20	0	0	0	0	S	0.0	0.0	0:00.05	kdevtmpfs
37	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-inet_
38	root	20	0	0	0	0	S	0.0	0.0	0:00.18	kauditd
39	root	20	0	0	0	0	S	0.0	0.0	0:00.00	khungtaskd
40	root	20	0	0	0	0	S	0.0	0.0	0:00.00	oom_reaper
42	root	20	0	0	0	0	I	0.0	0.0	0:00.18	kworker/u7:2-events_power_efficient
43	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-write
44	root	20	0	0	0	0	S	0.0	0.0	0:00.08	kcompactd0
45	root	25	5	0	0	0	S	0.0	0.0	0:00.00	ksmd
47	root	39	19	0	0	0	S	0.0	0.0	0:00.00	khugepaged
48	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kinte
ramesh@rameshserver:~\$ free -h											
	total		used		free		shared		buff/cache		available
Mem:	3.8Gi		429Mi		3.3Gi		1.1Mi		274Mi		3.4Gi
Swap:	3.8Gi		0B		3.8Gi						

Disk I/O Testing:

```

ramesh@rameshserver:~$ df if=dev/zero of=testfile bs=1G count=1 oflag=direct
df: 'if=dev/zero': No such file or directory
df: 'of=testfile': No such file or directory
df: 'bs=1G': No such file or directory
df: 'count=1': No such file or directory
df: 'oflag=direct': No such file or directory

```

Network Testing:

```

1073741824 bytes (1.1 GB, 1.0 GiB) Copied, 4.56051 S, 248 MB/s
ramesh@rameshserver:~$ iperf3 -s
-----
Server listening on 5201 (test #1)
-----
^Ciperf3: interrupt - the server has terminated
ramesh@rameshserver:~$ iperf3 -c ,server-ip>
-bash: syntax error near unexpected token `newline'
ramesh@rameshserver:~$ iperf3 -c <server-ip>
-bash: syntax error near unexpected token `newline'
ramesh@rameshserver:~$ prmot
Command 'prmot' not found, did you mean:
  command 'proot' from deb proot (5.1.0-1.3)
Try: sudo apt install <deb name>
ramesh@rameshserver:~$ ping <server-ip>
-bash: syntax error near unexpected token `newline'
ramesh@rameshserver:~$ iperf3 -c 192.168.56.101
iperf3: error - unable to connect to server - server may have stopped running or use a different port, firewall issue.
ramesh@rameshserver:~$ ping 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=1.26 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.094 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=18.8 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.097 ms
64 bytes from 192.168.56.101: icmp_seq=5 ttl=64 time=0.087 ms
64 bytes from 192.168.56.101: icmp_seq=6 ttl=64 time=0.059 ms
64 bytes from 192.168.56.101: icmp_seq=7 ttl=64 time=0.102 ms
64 bytes from 192.168.56.101: icmp_seq=8 ttl=64 time=0.086 ms
64 bytes from 192.168.56.101: icmp_seq=9 ttl=64 time=0.112 ms
64 bytes from 192.168.56.101: icmp_seq=10 ttl=64 time=0.114 ms
64 bytes from 192.168.56.101: icmp_seq=11 ttl=64 time=0.078 ms
64 bytes from 192.168.56.101: icmp_seq=12 ttl=64 time=0.123 ms
64 bytes from 192.168.56.101: icmp_seq=13 ttl=64 time=0.093 ms
64 bytes from 192.168.56.101: icmp_seq=14 ttl=64 time=0.117 ms
64 bytes from 192.168.56.101: icmp_seq=15 ttl=64 time=0.067 ms
64 bytes from 192.168.56.101: icmp_seq=16 ttl=64 time=0.081 ms
64 bytes from 192.168.56.101: icmp_seq=17 ttl=64 time=0.097 ms
^C
--- 192.168.56.101 ping statistics ---
17 packets transmitted, 17 received, 0% packet loss, time 17207ms
rtt min/avg/max/mdev = 0.059/1.261/18.780/4.388 ms

```

Network throughput tests were conducted using iperf3 from the workstation to the server. The ping test tool was also used to test the mean latency between the nodes in the network from the host-only network.

5. Conduct network performance analysis documenting latency and throughput?

Ans: Network Performance Measurement

Testing for Network Performance used iperf3 and ping.

Latency: Average round trip time: 1ms to 5ms on the host-only network

Throughput: Average of ~920 - 940 mbps which means there should be no to very minimal congestion on the network.

Packet Loss: 0% loss over the course of testing.

The virtual box host-only network provides the capability to create isolated and stable network connectivity; therefore, providing for secure testing and allowing for an SSH connection.

6. Capture optimisation analysis results describing improvements with quantitative data.?

Ans: To optimise performance, performance bottlenecks have been identified through stress tests. Stress tests showed that during stress tests, CPU was at maximum capacity and that the memory was under extreme pressure when multiple workloads were processed at the same time. It was also noticed that during a stress test, the SSH session was much less responsive when an application was being run under-load.

The first optimisation process to address CPU utilisation was to reduce the number of stress test threads being run concurrently as well as to improve the scheduling of workloads. After implementing this change, CPU utilisation dropped from around 100% to 85% and responsiveness increased.

The second optimisation process involved the optimisation of memory; background services that were not needed were removed and the way in which memory was allocated was improved. The result of these optimisations led to a reduction in the amount of memory used from 3.2 GB to 2.6 GB as well as to approximately 20% more available memory

Quantitative Improvement Summary:

metric	before	after	Improvement
CPU Utilisation	100%	85%	15%
Memory Usage	3.2 GB	2.6 GB	20%
SSH Response Time	100 ms	50 ms	50%
Disk Throughput	120 MB/s	150MB/s	25%